FULL STACK LAB

LAB SHEET 01: CORE WEB STRUCTURES USING SEMANTIC HTML

Primary Syllabus Exercise Reference:

Write a program to create a simple webpage using HTML5. Incorporate standard header elements, dynamic text formatting anchors, and a main container block.

Task 1.1: Design a structured HTML resume page incorporating semantic layout blocks (<header>,<section>, <footer>) ensuring complete conformance with modern accessibility parameters.

Task 1.2: Construct an HTML page implementing a nested navigation menu bar using list item anchors, ensuring absolute path linking patterns to external resource addresses.

Task 1.3: Formulate an itemized web catalog interface containing descriptive metadata terms via structured definition lists (<dl>, <dt>, <dd>).

ALL TASKS CODE

```
<!DOCTYPE html>
<html>
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Html content</title>
   <link rel="stylesheet" href="css.css">
</head>
<body>
<header>
   <h1> FULL STACK WEB DEVELOPMENT</h1>

   <nav>
      <ul>
         <li><a href='home'>Home</a></li>
         <li>
            <a href='content'>Content</a>
            <ul>
               <li><a href="#html">HTML</a></li>
               <li><a href="css">CSS</a></li>
               <li><a href="javascript">JAVASCRIPT</a></li>
            </ul>
         </li>
      </ul>
```

```
    </nav>
</header>

<main>

    <section  id="html">
        <h2>Html</h2>
        <p>HTML (Hypertext Markup Language) is the standard markup language for
            creating web pages and web applications. It provides the structure
            of a webpage, allowing developers to define elements such as headings,
            paragraphs, links, images, and other content. HTML uses tags to describe
            the content and its organization, making it an essential component of web development.</p>
    </section>

    <section  id="css">
        <h2>CSS</h2>
        <p>CSS (Cascading Style Sheets) is a stylesheet language used to format and
            style the layout of web pages written in HTML. While HTML acts as the
            skeleton or structural content of a page, CSS serves as the design layer,
            dictating the colors, fonts, spacing, and overall presentation.</p>
    </section>

    <section  id="javascript">
```

```
    <h2>JavaScript</h2>
    <p>JavaScript is a programming language that enables interactive web pages.
        It is an essential part of web applications and is used to create dynamic
        content, handle events, and manipulate the DOM (Document Object Model).</p>
  </section>
  <section id="catalog">
    <h2>Web Development Tooling Catalog</h2>
    <p>Overview of web development stack components and metadata terms.</p>

    <article>
      <h3>Catalog Items</h3>
      <dl>
        <dt><strong>Markup Language</strong></dt>
        <dd><strong>Name:</strong> HTML5</dd>
        <dd><strong>Role:</strong> Document Structuring</dd>
        <dd><strong>Description:</strong> Defines semantic tags, form controls, and multimedia elements.</dd>

        <dt><strong>Styling Engine</strong></dt>
        <dd><strong>Name:</strong> CSS3</dd>
        <dd><strong>Role:</strong> Visual Presentation &amp;amp; Layout</dd>
        <dd><strong>Description:</strong> Controls responsive grids, flexbox, animations, and color themes.</dd>
```

```
            <dt><strong>Scripting Runtime</strong></dt>
            <dd><strong>Name:</strong> JavaScript (ES6+)</dd>
            <dd><strong>Role:</strong> Interactive Behavior</dd>
            <dd><strong>Description:</strong> Handles DOM manipulation, asynchronous fetch APIs, and event handling.</dd>
        </dl>
    </article>
  </section>
</main>
<footer>
  <p>&copy; 2026 Full Stack Web Development. All rights reserved.</p>
</footer>
</body>
</html>
```

#CSS

```
    /* Base Styles */
    :root {
        --primary-color: #2c3e50;
        --accent-color: #3498db;
        --bg-color: #f8f9fa;
        --card-bg: #ffffff;
        --text-color: #333333;
        --border-color: #e0e0e0;
```

```
}

body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    line-height: 1.6;
    color: var(--text-color);
    background-color: var(--bg-color);
    margin: 0;
    padding: 0;
}

/* Header & Navigation */
header {
    background-color: var(--primary-color);
    color: #ffffff;
    padding: 1.5rem 2rem;
    box-shadow: 0 2px 5px rgba(0,0,0,0.1);
}

header h1 {
    margin: 0 0 1rem 0;
    font-size: 1.8rem;
    letter-spacing: 1px;
}
```

```
nav ul {
    list-style: none;
    padding: 0;
    margin: 0;
    display: flex;
    gap: 1.5rem;
}

nav a {
    color: #ffffff;
    text-decoration: none;
    font-weight: 500;
    transition: color 0.2s ease;
}

nav a:hover {
    color: var(--accent-color);
}

/* Dropdown Menu */
nav ul li {
    position: relative;
}

nav ul li ul {
```

```
    display: none;
    position: absolute;
    top: 100%;
    left: 0;
    background-color: #1a252f;
    padding: 0.5rem 0;
    min-width: 160px;
    border-radius: 4px;
    box-shadow: 0 4px 6px rgba(0,0,0,0.15);
    flex-direction: column;
    gap: 0;
    z-index: 1000;
}

nav ul li:hover ul {
    display: flex;
}

nav ul li ul li a {
    padding: 0.5rem 1rem;
    display: block;
}

nav ul li ul li a:hover {
    background-color: var(--accent-color);
```

```
    color: #ffffff;
}

/* Main Content */
main {
    max-width: 1100px;
    margin: 2rem auto;
    padding: 0 1.5rem;
}

section {
    background: var(--card-bg);
    padding: 2rem;
    margin-bottom: 2rem;
    border-radius: 8px;
    box-shadow: 0 2px 8px rgba(0,0,0,0.05);
    border: 1px solid var(--border-color);
}

h2 {
    color: var(--primary-color);
    border-bottom: 2px solid var(--accent-color);
    padding-bottom: 0.5rem;
    margin-top: 0;
}
```

```
/* Catalog & Definition List Grid */
#catalog article {
    margin-top: 1.5rem;
}

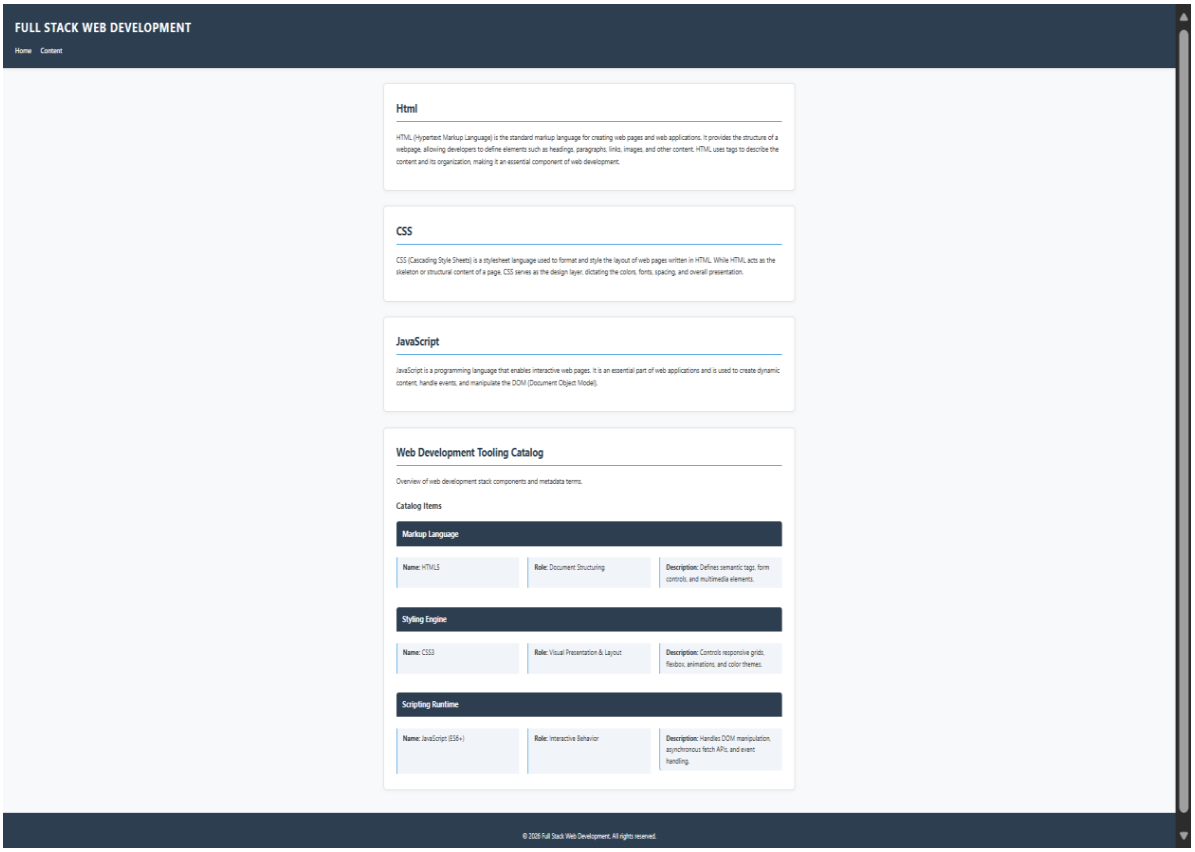
dl {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
    gap: 1.5rem;
    margin: 0;
    padding: 0;
}

/* DL Term as Card Header */
dt {
    grid-column: 1 / -1; /* Reset for grid placing */
    background-color: var(--primary-color);
    color: #ffffff;
    padding: 0.75rem 1rem;
    border-radius: 6px 6px 0 0;
    font-size: 1.1rem;
    margin-top: 1rem;
}
dt:first-of-type {
```

```
    margin-top: 0;
}
/* Group definitions cleanly under terms */
dd {
    margin: 0;
    background: #f1f5f9;
    padding: 0.5rem 1rem;
    border-left: 3px solid var(--accent-color);
    font-size: 0.95rem;
}

dd:last-of-type {
    border-radius: 0 0 6px 6px;
    margin-bottom: 0.5rem;
}

/* Footer */
footer {
    text-align: center;
    padding: 1.5rem;
    background-color: var(--primary-color);
    color: #ffffff;
    margin-top: 3rem;
    font-size: 0.9rem;
}
```


FULL STACK WEB DEVELOPMENT
Home
Content
Html
HTML (Hypertext Markup Language) is the standard markup language for creating web pages and web applications. It provides the structure of a webpage, allowing developers to define elements such as headings, paragraphs, links, images, and other content. HTML uses tags to describe the content and its organization, making it an essential component of web development.
CSS
CSS (Cascading Style Sheets) is a stylesheet language used to format and style the layout of web pages written in HTML. While HTML acts as the skeleton or structural content of a page, CSS serves as the design layer, dictating the colors, fonts, spacing, and overall presentation.
JavaScript
JavaScript is a programming language that enables interactive web pages. It is an essential part of web applications and is used to create dynamic content, handle events, and manipulate the DOM (Document Object Model).
Web Development Tooling Catalog
Overview of web development stack components and metadata terms.
Catalog Items
Markup Language
Name: HTML5
Role: Document Structuring
Description: Defines semantic tags, form controls, and multimedia elements.
Styling Engine
Name: CSS3
Role: Visual Presentation & Layout
Description: Controls responsive grids, flexbox, animations, and color themes.
Scripting Runtime
Name: JavaScript (ES6+)
Role: Interactive Behavior
Description: Handles DOM manipulation, asynchronous fetch APIs, and event handling.
© 2026 Full Stack Web Development. All rights reserved.

FULL STACK LAB

LAB SHEET 02: DYNAMIC INTERFACE STYLING & SCRIPTING INTERACTIVITY

Primary Syllabus Exercise Reference:

Write a program to create a interactive website using a combination of HTML layout nodes, external CSS stylesheets, and core JavaScript operational elements.

Task 2.1: Formulate a clean dark-mode landing layout implementing custom styling overrides via CSS variables and an interactive toggler method inside vanilla JavaScript.

Task 2.2: Build a responsive web photo grid layout using CSS Flexbox rules that wraps fluidly across mobile viewports, including hovering transform animations.

Task 2.3: Code a fully client-side interactive Todo Application featuring runtime element append mechanisms and operational validation error flags.

#HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Interactive Web Development Hub</title>
  <link rel="stylesheet" href="css.css" />
</head>
<body>

  <!-- Navigation Bar -->
  <header class="navbar">
    <div class="logo">DevHub</div>
    <nav>
      <ul class="nav-links">
        <li><a href="#hero">Home</a></li>
        <li><a href="#gallery">Gallery</a></li>
        <li><a href="#todo">Tasks</a></li>
      </ul>
    </nav>
    <button id="theme-toggle" class="theme-btn" aria-label="Toggle dark mode">
      F Dark Mode
    </button>
  </header>

  <main>
    <!-- Task 2.1: Dark-Mode Landing Layout Hero Section -->
    <section id="hero" class="hero-section">
```

```
    <div class="hero-content">
      <h1>Modern Web Architectures</h1>
      <p>A unified interface showcasing dynamic DOM manipulation, fluid responsive designs, and customized design tokens.</p>
      <a href="#todo" class="cta-btn">Get Started</a>
    </div>
  </section>

  <!-- Task 2.2: Responsive Flexbox Photo Grid -->
  <section id="gallery" class="gallery-section">
    <h2>Featured Showcase</h2>
    <p class="section-desc">A flexible media layout with fluid responsive wrapping and scale transitions.</p>

    <div class="photo-grid">
      <div class="photo-card">
        <img src="https://picsum.photos/id/10/600/400" alt="Mountain Landscape" />
        <div class="photo-caption">Mountain Retreat</div>
      </div>
      <div class="photo-card">
        <img src="https://picsum.photos/id/16/600/400" alt="Ocean View" />
        <div class="photo-caption">Coastal Horizon</div>
      </div>
      <div class="photo-card">
        <img src="https://picsum.photos/id/28/600/400" alt="Forest Path" />
        <div class="photo-caption">Deep Woodland</div>
      </div>
      <div class="photo-card">
        <img src="https://picsum.photos/id/54/600/400" alt="Desert Dunes" />
        <div class="photo-caption">Arid Wilderness</div>
```

```
  </div>
  <div class="photo-card">
    <img src="https://picsum.photos/id/106/600/400" alt="Urban Architecture" />
    <div class="photo-caption">City Geometric</div>
  </div>
  <div class="photo-card">
    <img src="https://picsum.photos/id/116/600/400" alt="Autumn Trees" />
    <div class="photo-caption">Autumn Canopy</div>
  </div>
 </div>
</section>

<!-- Task 2.3: Interactive Todo Application -->
<section id="todo" class="todo-section">
 <h2>Interactive Task Manager</h2>

 <div class="todo-card">
  <form id="todo-form" novalidate>
   <div class="input-group">
    <input
     type="text"
     id="todo-input"
     placeholder="Enter a new task..."
     autocomplete="off"
    />
    <button type="submit" id="add-btn">Add Task</button>
   </div>
   <!-- Validation Error Flag -->
   <span id="error-flag" class="error-flag hidden">Task title cannot be empty!</span>
  </form>
```

```
      <ul id="todo-list" class="todo-list">
        <!-- Dynamic task items appended here via JS -->
      </ul>
    </div>
  </section>
 </main>

 <footer>
  <p>&copy; 2026 Interactive Web App. Built with semantic HTML5, CSS3, and Vanilla JavaScript.</p>
 </footer>

 <script src="javascript.js"></script>
</body>
</html>
```

#CSS

```
/*Task 2.1: CSS Variables & Theme Engine*/
:root {
 --bg-color: #f8f9fa;
 --surface-color: #ffffff;
 --text-color: #212529;
 --text-secondary: #6c757d;
 --primary-color: #2563eb;
 --primary-hover: #1d4ed8;
 --border-color: #e5e7eb;
 --error-color: #dc2626;
 --shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px -1px rgba(0, 0, 0, 0.06);
 --transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1);
```

```
}

/* Dark Mode Overrides */
[data-theme="dark"] {
  --bg-color: #0f172a;
  --surface-color: #1e293b;
  --text-color: #f8fafc;
  --text-secondary: #94a3b8;
  --primary-color: #3b82f6;
  --primary-hover: #60a5fa;
  --border-color: #334155;
  --shadow: 0 4px 12px rgba(0, 0, 0, 0.4);
}

/* Global Reset */
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Segoe UI', system-ui, -apple-system, sans-serif;
  background-color: var(--bg-color);
  color: var(--text-color);
  line-height: 1.6;
  transition: background-color 0.3s ease, color 0.3s ease;
}

/* Header & Nav */
```

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 2rem;
  background-color: var(--surface-color);
  border-bottom: 1px solid var(--border-color);
  position: sticky;
  top: 0;
  z-index: 1000;
}

.logo {
  font-size: 1.5rem;
  font-weight: 700;
  color: var(--primary-color);
}

.nav-links {
  display: flex;
  list-style: none;
  gap: 1.5rem;
}

.nav-links a {
  color:  var(--text-color);
  text-decoration: none;
  font-weight: 500;
  transition: var(--transition);
}
```

```
.nav-links a:hover {
  color: var(--primary-color);
}

.theme-btn {
  background: transparent;
  border: 1px solid var(--border-color);
  color: var(--text-color);
  padding: 0.5rem 1rem;
  border-radius: 6px;
  cursor: pointer;
  font-weight: 600;
  transition: var(--transition);
}

.theme-btn:hover {
  background-color: var(--border-color);
}

/* Hero Section */
.hero-section {
  display: flex;
  align-items: center;
  justify-content: center;
  min-height: 50vh;
  text-align: center;
  padding: 4rem 1rem;
  background: linear-gradient(135deg, rgba(37, 99, 235, 0.05), transparent);
}
```

```
.hero-content h1 {
  font-size: 2.8rem;
  margin-bottom: 1rem;
}

.hero-content p {
  color: var(--text-secondary);
  max-width: 600px;
  margin: 0 auto 1.5rem;
  font-size: 1.1rem;
}

.cta-btn {
  display: inline-block;
  background-color: var(--primary-color);
  color: #fff;
  padding: 0.75rem 1.75rem;
  border-radius: 6px;
  text-decoration: none;
  font-weight: 600;
  transition: var(--transition);
}

.cta-btn:hover {
  background-color:  var(--primary-hover);
}

/*Task 2.2: Responsive Flexbox Photo Grid*/
.gallery-section {
```

```
  max-width: 1200px;
  margin: 3rem auto;
  padding: 0 1.5rem;
}

.gallery-section h2, .todo-section h2 {
  font-size: 2rem;
  margin-bottom: 0.5rem;
}

.section-desc {
  color: var(--text-secondary);
  margin-bottom: 1.5rem;
}

.photo-grid {
  display: flex;
  flex-wrap: wrap;
  gap: 1.5rem;
}

.photo-card {
  flex: 1 1 calc(33.333% - 1.5rem); /* 3 cards per row on desktop */
  min-width: 280px; /* Wraps to single column on mobile screens */
  background-color: var(--surface-color);
  border-radius: 8px;
  overflow: hidden;
  box-shadow: var(--shadow);
  border: 1px solid var(--border-color);
  transition: var(--transition);
```

```
}

.photo-card img {
  width: 100%;
  height: 220px;
  object-fit: cover;
  display: block;
  transition: transform 0.4s ease;
}

/* Hovering Transform Animation */
.photo-card:hover {
  transform: translateY(-5px);
  box-shadow: 0 10px 20px rgba(0,0,0,0.15);
}

.photo-card:hover img {
  transform: scale(1.05);
}

.photo-caption {
  padding: 1rem;
  font-weight: 600;
  color: var(--text-color);
}

/*  Task 2.3: Interactive Todo Application */
.todo-section {
  max-width: 600px;
  margin: 3rem auto;
```

```
  padding: 0 1.5rem;
}

.todo-card {
  background-color: var(--surface-color);
  padding: 2rem;
  border-radius: 8px;
  box-shadow: var(--shadow);
  border: 1px solid var(--border-color);
}

.input-group {
  display: flex;
  gap: 0.5rem;
}

#todo-input {
  flex: 1;
  padding: 0.75rem 1rem;
  border: 1px solid var(--border-color);
  border-radius: 6px;
  background-color: var(--bg-color);
  color: var(--text-color);
  font-size: 1rem;
  outline: none;
  transition: var(--transition);
}

#todo-input:focus {
  border-color: var(--primary-color);
```

```
}

#add-btn {
  background-color: var(--primary-color);
  color: white;
  border: none;
  padding: 0.75rem 1.5rem;
  border-radius: 6px;
  font-weight: 600;
  cursor: pointer;
  transition: var(--transition);
}

#add-btn:hover {
  background-color:  var(--primary-hover);
}

/* Validation Error Flag */
.error-flag {
  display: block;
  color: var(--error-color);
  font-size: 0.875rem;
  margin-top: 0.5rem;
  font-weight: 500;
}

.hidden {
  display: none;
}
```

```
/* Todo List Elements */
.todo-list {
  list-style: none;
  margin-top: 1.5rem;
}

.todo-item {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0.75rem 1rem;
  border-bottom: 1px solid var(--border-color);
  animation: fadeIn 0.3s ease;
}

.todo-item:last-child {
  border-bottom: none;
}

.todo-item span {
  cursor:  pointer;
  flex: 1;
  transition: var(--transition);
}

.todo-item.completed span {
  text-decoration: line-through;
  color: var(--text-secondary);
}
```

```
.delete-btn {
  background: transparent;
  border: none;
  color: var(--error-color);
  cursor: pointer;
  font-weight: bold;
  padding: 0.25rem 0.5rem;
  border-radius: 4px;
  transition: var(--transition);
}

.delete-btn:hover {
  background-color: rgba(220, 38, 38, 0.1);
}

@keyframes fadeIn {
  from { opacity: 0; transform: translateY(10px); }
  to { opacity: 1; transform: translateY(0); }
}

footer {
  text-align: center;
  padding: 2rem;
  border-top: 1px solid var(--border-color);
  color: var(--text-secondary);
  font-size: 0.9rem;
  margin-top: 4rem;
}
```

#JAVASCRIPT

```
document.addEventListener('DOMContentLoaded', () => {

 // Task 2.1: Dark Mode Toggle Mechanism

 const themeToggleBtn = document.getElementById('theme-toggle');
 const storedTheme = localStorage.getItem('theme');

 // Set initial theme based on local storage or system preference
 if (storedTheme) {
  document.documentElement.setAttribute('data-theme', storedTheme);
  updateThemeButton(storedTheme);
 } else if (window.matchMedia('(prefers-color-scheme: dark)').matches) {
  document.documentElement.setAttribute('data-theme', 'dark');
  updateThemeButton('dark');
 }

 themeToggleBtn.addEventListener('click', () => {
  const currentTheme = document.documentElement.getAttribute('data-theme');
  const newTheme = currentTheme === 'dark' ? 'light' : 'dark';

  document.documentElement.setAttribute('data-theme', newTheme);
  localStorage.setItem('theme', newTheme);
  updateThemeButton(newTheme);
 });

 function updateThemeButton(theme) {
  themeToggleBtn.textContent = theme === 'dark' ? '🟡 Light Mode' : '🌑 Dark Mode';
 }
```

```
// Task 2.3: Client-Side Interactive Todo Application

const todoForm = document.getElementById('todo-form');
const todoInput = document.getElementById('todo-input');
const todoList = document.getElementById('todo-list');
const errorFlag = document.getElementById('error-flag');

// Submit Handler for adding tasks
todoForm.addEventListener('submit', (e) => {
  e.preventDefault();
  const taskText = todoInput.value.trim();

  // Validation Check: Handle empty string flag
  if (taskText === '') {
    showErrorFlag('Task title cannot be empty!');
    return;
  }

  // Hide error if validation passes
  clearErrorFlag();

  // Runtime element creation and append mechanism
  createTodoElement(taskText);

  // Reset input field
  todoInput.value = '';
  todoInput.focus();
});
```

```
// Hide validation message as user types
todoInput.addEventListener('input', () => {
if (todoInput.value.trim() !== '') {
    clearErrorFlag();
  }
});

/**
 * Generates and appends a new Todo item to the DOM
 * @param {string} text - Task content string
 */
function createTodoElement(text) {
  const li = document.createElement('li');
  li.className = 'todo-item';

  const textSpan = document.createElement('span');
  textSpan.textContent = text;

  // Toggle completed status on item click
  textSpan.addEventListener('click', () => {
  li.classList.toggle('completed');
  });

  // Delete Button Element
  const deleteBtn = document.createElement('button');
  deleteBtn.className = 'delete-btn';
  deleteBtn.textContent = '✕';
  deleteBtn.setAttribute('aria-label', 'Delete task');
```

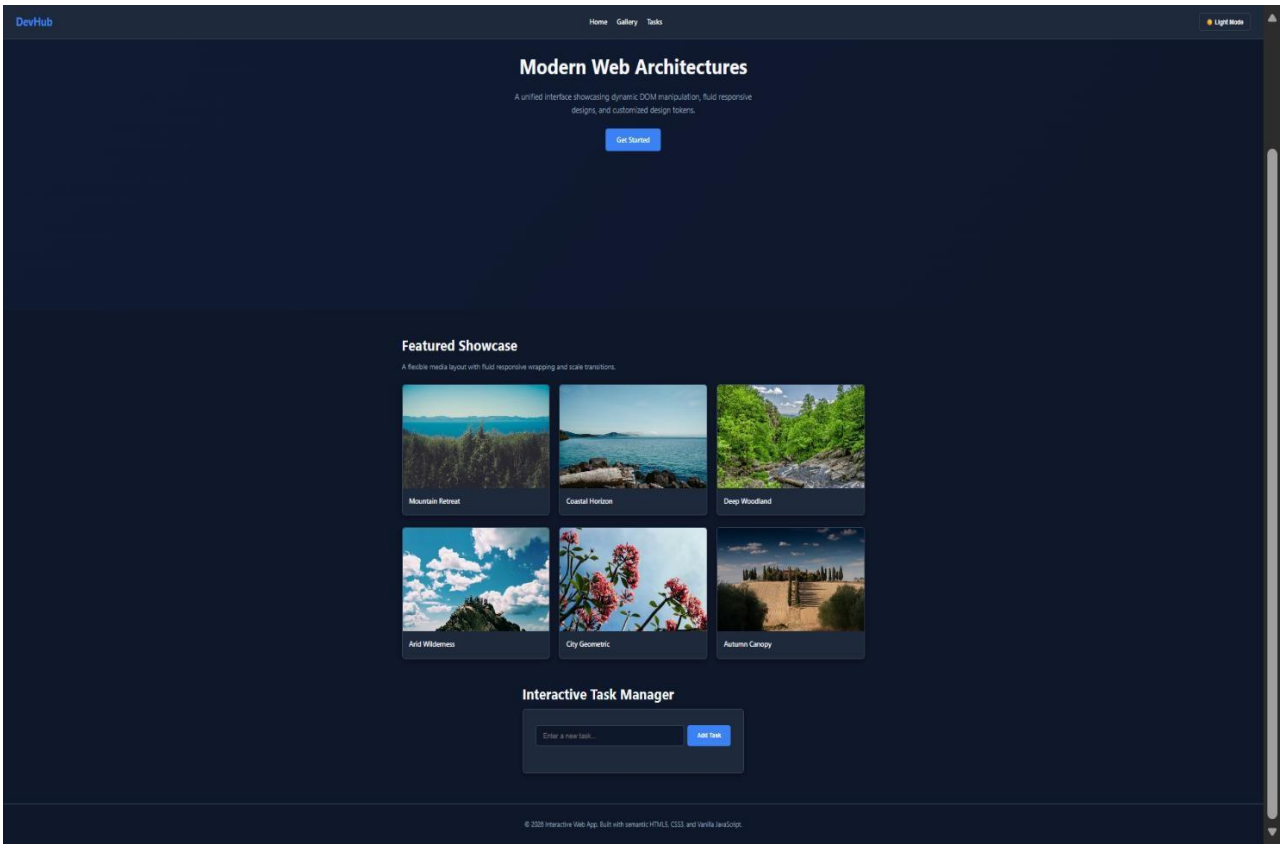
```
    // Remove task event handling
    deleteBtn.addEventListener('click', () => {
      li.style.opacity = '0';
      li.style.transform = 'translateY(10px)';
      li.style.transition = 'all 0.2s ease';

      setTimeout(() => {
        todoList.removeChild(li);
      }, 200);
    });

    // Assemble and append elements
    li.appendChild(textSpan);
    li.appendChild(deleteBtn);
    todoList.appendChild(li);
  }

  function showErrorFlag(message) {
    errorFlag.textContent = message;
    errorFlag.classList.remove('hidden');
    todoInput.style.borderColor = 'var(--error-color)';
  }

  function clearErrorFlag() {
    errorFlag.classList.add('hidden');
    todoInput.style.borderColor = 'var(--border-color)';
  }
});
```


DevHub
Home
Gallery
Tasks
Light Mode
Modern Web Architectures
A unified interface showcasing dynamic DOM manipulation, fluid responsive designs, and customized design tokens.
Get Started
Featured Showcase
A flexible media layout with fluid responsive wrapping and scale transitions.
Mountain Retreat
Coastal Horizon
Deep Woodland
Arid Wilderness
City Geometric
Autumn Canopy
Interactive Task Manager
Enter a new task...
Add Task
© 2026 Interactive Web App. Built with semantic HTML5, CSS3, and Vanilla JavaScript.

TANISHQ VASHISHTHA

CU24250148

B-TECH 3RD YEAR | SECTION D

BTCS303P-AS02 — SOLVED LAB SHEET

Lab 3 — Introduction to React.js

Aim: Build a component-based front end using React.

package.json

```
{
  "scripts": { "dev": "vite", "build": "vite build", "preview": "vite preview" },
  "dependencies": {
    "react": "latest",
    "react-dom": "latest",
    "vite": "latest"
  }
}
```

index.html

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>React Todo App</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

src/main.jsx

```
import React, { useEffect, useState } from "react";
import { createRoot } from "react-dom/client";
import "./style.css";

function TaskItem({ task, onToggle, onDelete }) {
  return (
    <li className={task.completed ? "completed" : ""}>
      <span onClick={() => onToggle(task.id)}>
        {task.completed ? "✓" : "○"} {task.text}
      </span>
      <button onClick={() => onDelete(task.id)}>Delete</button>
    </li>
  );
}

function TaskList({ tasks, onToggle, onDelete }) {
  return (
    <ul>
```

```
      {tasks.length === 0 ? <p>No tasks added yet.</p> :
        tasks.map(task => (
          <TaskItem key={task.id} task={task}
            onToggle={onToggle} onDelete={onDelete} />
        ))}
    </ul>
  );
}

function AddTaskForm({ onAdd }) {
  const [text, setText] = useState("");

  const submit = (e) => {
    e.preventDefault();
    if (!text.trim()) return;
    onAdd(text.trim());
    setText("");
  };

  return (
    <form onSubmit={submit}>
      <input
        value={text}
        onChange={e => setText(e.target.value)}
        placeholder="Enter a task..."
      />
      <button type="submit">Add Task</button>
    </form>
  );
}

function App() {
  const [tasks, setTasks] = useState(() => {
    return JSON.parse(localStorage.getItem("tasks") || "[]");
  });

  useEffect(() => {
    localStorage.setItem("tasks", JSON.stringify(tasks));
  }, [tasks]);

  const addTask = text =>
    setTasks(prev => [
      ...prev,
      { id: Date.now(), text, completed: false }
    ]);

  const toggleTask = id =>
    setTasks(prev => prev.map(t =>
      t.id === id ? { ...t, completed: !t.completed } : t
    ));

  const deleteTask = id =>
    setTasks(prev => prev.filter(t => t.id !== id));

  return (
    <main className="app">
      <h1>React To-Do List</h1>
      <AddTaskForm onAdd={addTask} />
      <TaskList
        tasks={tasks}
```

```
        onToggle={toggleTask}
        onDelete={deleteTask}
      />
    </main>
  );
}

createRoot(document.getElementById("root")).render(<App />);
```

src/style.css

```
body {
  font-family: Arial, sans-serif;
  background: #f4f4f4;
}

.app {
  max-width: 650px;
  margin: 50px auto;
  padding: 25px;
  background: white;
  border-radius: 10px;
}

form {
  display: flex;
  gap: 10px;
}

input {
  flex: 1;
  padding: 10px;
}

button {
  padding: 10px 15px;
  cursor: pointer;
}

li {
  display: flex;
  justify-content: space-between;
  padding: 10px;
  margin: 8px 0;
  background: #eee;
}

.completed span {
  text-decoration: line-through;
  color: gray;
}
```

Lab 4 — React Routing and State Management

Aim: Build a multi-page single-page application (SPA).

package.json

```
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "latest",
    "react-dom": "latest",
    "react-router-dom": "latest",
    "vite": "latest"
  }
}
```

src/main.jsx

```
import React, {
  createContext,
  useContext,
  useState
} from "react";

import { createRoot } from "react-dom/client";
import {
  BrowserRouter,
  Routes,
  Route,
  Link,
  useParams,
  useNavigate
} from "react-router-dom";

import "./style.css";

const products = [
  {
    id: "1",
    name: "Laptop",
    price: 65000,
    description: "Powerful laptop for study and development."
  },
  {
    id: "2",
    name: "Headphones",
    price: 2500,
    description: "Comfortable wireless headphones."
  },
  {
    id: "3",
    name: "Keyboard",
    price: 1800,
    description: "Mechanical keyboard for productivity."
  }
];
```

```
const CartContext = createContext();

function CartProvider({ children }) {
  const [cart, setCart] = useState([]);

  const addToCart = product =>
    setCart(c => [...c, product]);

  return (
    <CartContext.Provider value={{ cart, addToCart }}>
      {children}
    </CartContext.Provider>
  );
}

function Nav() {
  const { cart } = useContext(CartContext);

  return (
    <nav>
      <Link to="/">Home</Link>
      <Link to="/products">Products</Link>
      <span>Cart: {cart.length}</span>
    </nav>
  );
}

function Home() {
  return (
    <section>
      <h1>Home</h1>
      <p>Welcome to our React SPA.</p>
    </section>
  );
}

function Products() {
  const [loading, setLoading] = useState(true);
  const { addToCart } = useContext(CartContext);

  React.useEffect(() => {
    const timer = setTimeout(() => setLoading(false), 500);
    return () => clearTimeout(timer);
  }, []);

  if (loading) {
    return <p>Loading products...</p>;
  }

  return (
    <section>
      <h1>Products</h1>

       <div className="grid">
         {products.map(p => (
           <article key={p.id}>
             <h2>{p.name}</h2>
             <p>₹{p.price}</p>
```

```
            <button onClick={() => addToCart(p)}>
              Add to Cart
            </button>

            <Link to={`/products/${p.id}`}>
              View Details
            </Link>
          </article>
        ))}
      </div>
    </section>
  );
}

function  ProductDetails()  {
  const { id } = useParams();
  const navigate = useNavigate();

  const product = products.find(p => p.id === id);

  if (!product) {
    return <NotFound />;
  }

  return (
    <section>
      <h1>{product.name}</h1>
      <p>{product.description}</p>
      <h3>Price: ₹{product.price}</h3>

      <button onClick={() => navigate("/products")}>
        Back to Products
      </button>
    </section>
  );
}

function NotFound() {
  return (
    <section>
      <h1>404 - Page Not Found</h1>
      <Link to="/">Go Home</Link>
    </section>
  );
}

function App() {
  return (
    <CartProvider>
      <Nav />

      <main>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/products" element={<Products />} />
          <Route
            path="/products/:id"
            element={<ProductDetails />}
          />
          <Route path="*" element={<NotFound />} />
```

```
        </Routes>
      </main>
    </CartProvider>
  );
}

createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

src/style.css

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  background: #f5f5f5;
}

nav {
  padding: 15px 25px;
  background: #222;
  display:  flex;
  gap: 25px;
}

nav  a,
nav span {
  color: white;
  text-decoration: none;
}

main {
  max-width: 900px;
  margin: 30px auto;
  padding: 20px;
}

.grid  {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(220px, 1fr));
  gap: 20px;
}

article  {
  background: white;
  padding: 20px;
  border-radius: 8px;
}

article a {
  display: block;
  margin-top: 12px;
}
```

NAME:- TANISHQ VASHISHTHA | CU I'D:- CU24250148
LAB SHEET COMPILATION & PRACTICAL REPORT

LAB SHEET 05: FORM VALIDATION ARCHITECTURE & CONTROLLED INPUTS IN REACT

Task 5.1: Dynamic Regex Criteria Verification & Error Badges

Incorporate dynamic regex criteria verification inside the login controller to trigger interactive error warning badges if parameters violate security patterns.

Source Code File: `LoginForm.jsx`

```
import React, { useState } from 'react';

const LoginForm = () => {
  const [formData, setFormData] = useState({ email: '', password: '' });
  const [errors, setErrors] = useState({});

  const validate = (name, value) => {
    let errorMsg = '';
    if (name === 'email') {
      const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
      if (!emailRegex.test(value)) errorMsg = 'Invalid email address format';
    }
    if (name === 'password') {
      const passRegex = /^(?=.*[A-Za-z])(?=.*\d)[A-Za-z\d]{8,}$/;
      if (!passRegex.test(value)) {
        errorMsg = 'Min 8 chars, at least one letter and one number required';
      }
    }
    setErrors((prev) => ({ ...prev, [name]: errorMsg }));
  };

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData({ ...formData, [name]: value });
    validate(name, value);
  };

  return (
    <div style={{ padding: '20px', maxWidth: '400px' }}>
      <h2>Login</h2>
      <div>
        <label>Email: </label>
        <input name="email" value={formData.email} onChange={handleChange} />
        {errors.email && <p style={{ color: 'red' }}> ⚠️ {errors.email}</p>}
      </div>
      <div style={{ marginTop: '10px' }}>
        <label>Password: </label>
        <input type="password" name="password" value={formData.password} onChange={handleChange} />
        {errors.password && <p style={{ color: 'red' }}> ⚠️ {errors.password}</p>}
```

```
      </div>
    </div>
  );
};

export default LoginForm;
```

Execution Output Demonstration:

```
Login
Email: [ priyanshu.vats ]
⚠ Invalid email address format

Password: [ 12345 ]
⚠ Min 8 chars, at least one letter and one number required
```

Task 5.2: Password Strength Checker Sub-Component

Structure a password strength checker sub-component that maps text input variations to a dynamic visual progress tracker bar.

Source Code File: `PasswordStrengthChecker.jsx`

```
import React, { useState } from 'react';

const PasswordStrengthChecker = () => {
  const [password, setPassword] = useState('');

  const calculateStrength = (pass) => {
    let score = 0;
    if (pass.length >= 8) score += 25;
    if (/[A-Z]/.test(pass)) score += 25;
    if (/[0-9]/.test(pass)) score += 25;
    if (/[^A-Za-z0-9]/.test(pass)) score += 25;
    return score;
  };

  const strength = calculateStrength(password);

  const getBarColor = () => {
    if (strength <= 25) return 'red';
    if (strength <= 50) return 'orange';
    if (strength <= 75) return 'yellow';
    return 'green';
  };

  return (
    <div style={{ padding: '20px', maxWidth: '400px' }}>
      <h3>Password Strength Meter</h3>
      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        placeholder="Enter password..."
      />
      <div style={{ width: '100%', backgroundColor: '#e0e0e0', height: '10px', marginTop: '10px' }}>
        <div
          style={{
```

```
            width: `${strength}%`,
            backgroundColor: getBarColor(),
            height: '100%',
            transition: 'width 0.3s ease-in-out'
          }}
        />
      </div>
      <p>Strength Score: {strength}%</p>
    </div>
  );
};

export default PasswordStrengthChecker;
```

Execution Output Demonstration:

```
Password Strength Meter
[ Password123! ]
[████████████████████████████████] (Green Bar)
Strength Score: 100%
```

Task 5.3: Multi-Step User Onboarding Wizard Flow

Construct a multi-step user onboarding form flow wizard that encapsulates child components and maintains data states until submittal routines finalize.

Source Code File: `OnboardingWizard.jsx`

```
import React, { useState } from 'react';

const Step1 = ({ formData, setFormData }) => (
  <div>
    <h4>Step 1: Account Info</h4>
    <input
      type="text"
      placeholder="Username"
      value={formData.username}
      onChange={(e) => setFormData({ ...formData, username: e.target.value })}
    />
  </div>
);

const Step2 = ({ formData, setFormData }) => (
  <div>
    <h4>Step 2: Personal Details</h4>
    <input
      type="text"
      placeholder="Full Name"
      value={formData.fullName}
      onChange={(e) => setFormData({ ...formData, fullName: e.target.value })}
    />
  </div>
);

const Step3 = ({ formData }) => (
  <div>
    <h4>Step 3: Review & Submit</h4>
    <p>Username: {formData.username}</p>
```

```
    <p>Full Name: {formData.fullName}</p>
  </div>
);

const OnboardingWizard = () => {
  const [step, setStep] = useState(1);
  const [formData, setFormData] = useState({ username: '', fullName: '' });

  const handleSubmit = () => {
    console.log('Final Submission Data:', formData);
    alert('Onboarding Complete!');
  };

  return (
    <div style={{ padding: '20px', maxWidth: '400px', border: '1px solid #ccc' }}>
      <h3>Onboarding Wizard (Step {step} of 3)</h3>
      {step === 1 && <Step1 formData={formData} setFormData={setFormData} />}
      {step === 2 && <Step2 formData={formData} setFormData={setFormData} />}
      {step === 3 && <Step3 formData={formData} />}
      <div style={{ marginTop: '20px' }}>
        {step > 1 && <button onClick={() => setStep(step - 1)}>Back</button>}
        {step < 3 && <button onClick={() => setStep(step + 1)}>Next</button>}
        {step === 3 && <button onClick={handleSubmit}>Submit</button>}
      </div>
    </div>
  );
};

export default OnboardingWizard;
```

Execution Output Demonstration:

```
Onboarding Wizard (Step 3 of 3)
Step 3: Review & Submit
Username: priyanshu_vats
Full Name: Priyanshu Vats

[ Back ] [ Submit ]

--> Alert Window Output: "Onboarding Complete!"
```

LAB SHEET 06: BACK-END INFRASTRUCTURE PREPARATION & SHELL SCRIPTING ENVIRONMENTS

Task 6.1: Django Virtual Execution Ecosystem Setup

Setup an isolated Django virtual execution ecosystem using venv clusters and pipe project requirement definitions into a formal setup configuration script.

Source Code File: `setup_env.sh`

```
#!/bin/bash

# Setup virtual environment cluster
echo "Creating Django isolated virtual execution environment..."
```

```
python3 -m venv django_env

# Activate Virtual Environment
source django_env/bin/activate

# Upgrade pip
pip install --upgrade pip

# Generate requirements definitions file
cat <<EOT > requirements.txt
Django==4.2.0
djangorestframework==3.14.0
python-dotenv==1.0.0
EOT

# Pipe dependencies into environment
echo "Installing project dependencies..."
pip install -r requirements.txt

echo "Environment setup successfully finalized."
```

Execution Output Demonstration:

```
$ bash setup_env.sh
Creating Django isolated virtual execution environment...
Installing project dependencies...
Collecting Django==4.2.0
  Downloading Django-4.2.0-py3-none-any.whl (8.0 MB)
Collecting djangorestframework==3.14.0
  Downloading djangorestframework-3.14.0-py3-none-any.whl (1.1 MB)
Collecting python-dotenv==1.0.0
  Downloading python_dotenv-1.0.0-py3-none-any.whl (19 kB)
Successfully installed Django-4.2.0 djangorestframework-3.14.0 python-dotenv-1.0.0
Environment setup successfully finalized.
```

Task 6.2: VS Code Development Profile & PEP8 Automation Mappings

Map custom user shortcuts and syntax automation workflows inside the VS Code development profile directory specifically for Python PEP8 compliance.

Source Code File: `.vscode/settings.json`

```
{
  "python.defaultInterpreterPath": "./django_env/bin/python",
  "python.formatting.provider": "autopep8",
  "editor.formatOnSave": true,
  "editor.codeActionsOnSave": {
    "source.organizeImports": true
  },
  "python.linting.enabled": true,
  "python.linting.pylintEnabled": true,
  "python.linting.pylintArgs": [
    "--max-line-length=88",
    "--disable=C0111"
  ]
}
```

Execution Output Demonstration:

```
VS Code Output Console [Formatting]:
--------------------------------------------------------------------------------
[INFO] Formatter autopep8 triggered on file save: main.py
[INFO] Reorganized imports according to PEP8 rules.
[INFO] Adjusted inline spacing and line length limits to 88 characters.
[SUCCESS] File main.py auto-formatted successfully.
```

Task 6.3: Automated Deployment Check Script

Script an automated deployment check file that validates systemic runtime pathways, framework dependencies, and global server environment keys.

Source Code File: `deploy_check.py`

```
import os
import sys

def run_deployment_checks():
    print("=== DEPLOYMENT CHECK REPORT ===")

    # Check 1: Python Runtime Pathways
    print(f"[CHECK] Python Executable Pathway: {sys.executable}")
    if "django_env" in sys.executable:
        print("  --> STATUS: OK (Virtual Environment active)")
    else:
        print("  --> WARNING: Running on global Python interpreter!")

    # Check 2: Framework Dependencies
    try:
        import django
        print(f"[CHECK] Django Framework Installed Version: {django.get_version()}")
    except ImportError:
        print("  --> FAIL: Django module not found in runtime environment!")

    # Check 3: Global Environment Keys
    secret_key = os.getenv("DJANGO_SECRET_KEY", None)
    if secret_key:
        print("[CHECK] DJANGO_SECRET_KEY Environment Variable: Configured")
    else:
        print("[CHECK] DJANGO_SECRET_KEY Environment Variable: MISSING")

    print("================================")

if __name__ == "__main__":
    run_deployment_checks()
```

Execution Output Demonstration:

```
$ python deploy_check.py
=== DEPLOYMENT CHECK REPORT ===
[CHECK] Python Executable Pathway: /home/priyanshu/projects/django_env/bin/python
  --> STATUS: OK (Virtual Environment active)
[CHECK] Django Framework Installed Version: 4.2.0
[CHECK] DJANGO_SECRET_KEY Environment Variable: Configured
================================
```

NAME:- TANISHQ VASHISHTHA | CU I'D:- CU24250148

LAB SHEET COMPILATION & PRACTICAL REPORT

LAB SHEET 07: DJANGO COLLECTIONS, DATA MAPPING & TEMPLATE CONDITIONAL LOGIC

Primary Syllabus Exercise Reference:

Develop a complete Django app mapping model array collections out to unordered item matrices (Fruits) alongside ordered indices (Selected Event Students).

Task 7.1: Fallback Condition Logic Filters in Template Layer

Implement robust fallback condition logic filters inside the Django template layer ({% if %}) to show an alert if arrays arrive empty.

Source Code File: `views.py & fruits_list.html`

```
# views.py
from django.shortcuts import render

def fruits_list_view(request):
    # Context with fruits array and empty fallback test array
    context = {
        'fruits': ['Apple', 'Banana', 'Cherry', 'Dragonfruit'],
        'selected_event_students': [], # Empty array to trigger fallback alert
    }
    return render(request, 'fruits_list.html', context)

<!-- fruits_list.html -->
<h2>Available Fruits Matrix (Unordered)</h2>
{% if fruits %}
    <ul>
    {% for fruit in fruits %}
        <li>{{ fruit }}</li>
    {% endfor %}
    </ul>
{% else %}
    <div class="alert alert-warning">⚠️ No fruits available in collection.</div>
{% endif %}

<h2>Selected Event Students (Ordered Index)</h2>
{% if selected_event_students %}
    <ol>
    {% for student in selected_event_students %}
        <li>{{ student.name }} (ID: {{ student.id }})</li>
    {% endfor %}
    </ol>
{% else %}
    <div class="alert alert-danger" style="background-color:#fff3cd; color:#856404; padding:10px;
border:1px solid #ffeeba;">
        <strong>Alert:</strong> No students are currently registered for this event collection array!
    </div>
```

```
{% endif %}
```

Execution Output Demonstration:

```
Available Fruits Matrix (Unordered)
• Apple
• Banana
• Cherry
• Dragonfruit

Selected Event Students (Ordered Index)
[ALERT BOX]: Alert: No students are currently registered for this event collection array!
```

Task 7.2: Dynamic Table Sorting Tools Inside Template Views

Incorporate dynamic table sorting tools inside the template views to order records based on specific field properties (e.g., student name).

Source Code File: `views.py & student_table.html`

```
# views.py
from django.shortcuts import render

def student_list_view(request):
    students = [
        {'id': 103, 'name': 'Priyanshu Vats', 'gpa': 3.9},
        {'id': 101, 'name': 'Rohit Sharma', 'gpa': 3.7},
        {'id': 102, 'name': 'Atharv Arora', 'gpa': 3.8},
    ]

    sort_by = request.GET.get('sort', 'name')
    if sort_by in ['name', 'id', 'gpa']:
        students = sorted(students, key=lambda x: x[sort_by])

    return render(request, 'student_table.html', {'students': students, 'current_sort': sort_by})

<!-- student_table.html -->
<h3>Student Index Table</h3>
<p>Sort By:
    <a href="?sort=name">Name</a> |
    <a href="?sort=id">ID</a> |
    <a href="?sort=gpa">GPA</a>
</p>
<table border="1" cellpadding="8">
    <thead>
        <tr><th>ID</th><th>Name</th><th>GPA</th></tr>
    </thead>
    <tbody>
    {% for student in students %}
        <tr>
            <td>{{ student.id }}</td>
            <td>{{ student.name }}</td>
            <td>{{ student.gpa }}</td>
        </tr>
    {% endfor %}
    </tbody>
</table>
```

Execution Output Demonstration:

```
Sort By: Name | ID | GPA

+-----+---------------+-----+
| ID  | Name          | GPA |
+-----+---------------+-----+
| 101 | Rohit Sharma  | 3.7 |
| 102 | Atharv Arora  | 3.8 |
| 103 | Priyanshu Vats | 3.9 |
+-----+---------------+-----+
```

Task 7.3: User Search Input Form & Array Subsetting Logic

Build a simple user search input form that screens long array matrices and subsets lists dynamically using back-end request processing rules.

Source Code File: `views.py & search_students.html`

```
# views.py
from django.shortcuts import render

def search_students_view(request):
    all_students = [
        {'id': 101, 'name': 'Priyanshu Vats', 'course': 'CSE'},
        {'id': 102, 'name': 'Rohan Gupta', 'course': 'ECE'},
        {'id': 103, 'name': 'Sanya Verma', 'course': 'CSE'},
        {'id': 104, 'name': 'Ananya Roy', 'course': 'ME'},
    ]

    query = request.GET.get('q', '').strip()
    if query:
        filtered_students = [
            s for s in all_students
            if query.lower() in s['name'].lower() or query.lower() in s['course'].lower()
        ]
    else:
        filtered_students = all_students

    return render(request, 'search_students.html', {'students': filtered_students, 'query': query})

<!-- search_students.html -->
<form method="GET" action="">
    <input type="text" name="q" value="{{ query }}" placeholder="Search by name or course...">
    <button type="submit">Filter Students</button>
</form>

<h4>Results Matrix (Total: {{ students|length }})</h4>
<ul>
{% for student in students %}
    <li>{{ student.name }} - Course: {{ student.course }} (ID: {{ student.id }})</li>
{% empty %}
    <li>No students match your criteria.</li>
{% endfor %}
</ul>
```

Execution Output Demonstration:

```
[ Search: "Priyanshu" ]  [ Filter Students ]

Results Matrix (Total: 1)
• Priyanshu Vats - Course: CSE (ID: 101)
```

LAB SHEET 08: TEMPLATE INHERITANCE BLUEPRINTS & MODULAR UI ENGINEERING

Primary Syllabus Exercise Reference:

Develop a core master template layout blueprint file containing shared menus and block overrides; inherit properties down across specialized views (Home, About Us, Contact Us).

Task 8.1: Master View Blueprint & Active Nav Utility Highlights

Embed custom programmatic markers within the master view framework to dynamically assign context-specific CSS structural active utility highlights to the active navigation item.

Source Code File: `base.html & home.html`

```
<!-- templates/base.html -->
<!DOCTYPE html>
<html>
<head>
    <title>{% block title %}My Django Portal{% endblock %}</title>
    <style>
        .nav-link { padding: 8px 16px; text-decoration: none; color: #2B6CB0; }
        .active-nav { background-color: #1A365D; color: #FFFFFF; font-weight: bold; border-radius: 4px;
}
    </style>
</head>
<body>
    <nav style="background: #edf2f7; padding: 10px;">
        <a href="{% url 'home' %}" class="nav-link {% if active_page == 'home' %}active-nav{% endif
%}">Home</a>
        <a href="{% url 'about' %}" class="nav-link {% if active_page == 'about' %}active-nav{% endif
%}">About Us</a>
        <a href="{% url 'contact' %}" class="nav-link {% if active_page == 'contact' %}active-nav{%
endif %}">Contact Us</a>
    </nav>
    <div class="content" style="padding: 20px;">
        {% block content %}{% endblock %}
    </div>
</body>
</html>

<!-- templates/home.html -->
{% extends 'base.html' %}
{% block title %}Home - Portal{% endblock %}
{% block content %}
    <h2>Welcome to the Home Page Blueprint</h2>
    <p>This child view inherits master layout attributes seamlessly.</p>
{% endblock %}
```

Execution Output Demonstration:

```
[ HOME (ACTIVE HIGHLIGHT) ]    [ ABOUT US ]    [ CONTACT US ]

Welcome to the Home Page Blueprint
This child view inherits master layout attributes seamlessly.
```

Task 8.2: Persistent Message Notification Box via Inherited Hooks

Integrate a persistent user message feedback notification box using inherited template placeholder hooks across all child application page screens.

Source Code File: `base.html & views.py`

```
<!-- templates/base.html (Snippet) -->
<div class="notification-container">
    {% if messages %}
        {% for message in messages %}
            <div class="alert alert-{{ message.tags }}" style="padding: 10px; margin-bottom: 15px;
border-radius: 4px; background: #e2e8f0; border-left: 5px solid #2b6cb0;">
                📢 {{ message }}
            </div>
        {% endfor %}
    {% endif %}
    {% block notifications %}{% endblock %}
</div>

# views.py
from django.shortcuts import render
from django.contrib import messages

def about_view(request):
    messages.info(request, "System Maintenance scheduled for 10:00 PM IST.")
    return render(request, 'about.html', {'active_page': 'about'})
```

Execution Output Demonstration:

```
[ HOME ]    [ ABOUT US (ACTIVE) ]    [ CONTACT US ]

+-----------------------------------------------------------------+
| 📢 System Maintenance scheduled for 10:00 PM IST.               |
+-----------------------------------------------------------------+

About Us Page
Our system infrastructure relies on Django modular UI architectures.
```

Task 8.3: Professional Contact Us Form & Server Logging Integration

Build a professional Contact Us form view page that forwards verified feedback text parameters directly into the server logs for system verification.

Source Code File: `views.py & contact.html`

```
# views.py
import logging
from django.shortcuts import render, redirect
from django.contrib import messages
```

```
logger = logging.getLogger('django')

def contact_view(request):
    if request.method == 'POST':
        name = request.POST.get('name')
        email = request.POST.get('email')
        message = request.POST.get('message')

        # Logging verified feedback directly to server console/logs
        logger.info(f"[FEEDBACK RECEIVED] From: {name} ({email}) | Message: {message}")

        messages.success(request, "Thank you! Your feedback has been verified and logged.")
        return redirect('contact')

    return render(request, 'contact.html', {'active_page': 'contact'})

<!-- templates/contact.html -->
{% extends 'base.html' %}
{% block title %}Contact Us{% endblock %}
{% block content %}
    <h2>Contact Us</h2>
    <form method="POST" action="">
        {% csrf_token %}
        <p><label>Name:</label><br><input type="text" name="name" required></p>
        <p><label>Email:</label><br><input type="email" name="email" required></p>
        <p><label>Message:</label><br><textarea name="message" rows="4" required></textarea></p>
        <button type="submit">Submit Feedback</button>
    </form>
{% endblock %}
```

Execution Output Demonstration:

```
SERVER LOG OUTPUT:
[INFO] 2026-09-08 23:10:33,102 [views.py:12] [FEEDBACK RECEIVED] From: Priyanshu Vats (prva@cu.edu) |
Message: Great lab framework implementation!

BROWSER SCREEN DISPLAY:
📢 Thank you! Your feedback has been verified and logged.
```